

Authors:

Jake Crawford, Caleb Neil, Lena Palmieri

The meaning of our Turing Machine is to determine whether the given input of the string belongs to the language of the unary prime numbers. So in this assignment, the language that we are working with is that it is made up of strings that are over the alphabet $\{1\}$, where the length of n in a unary string represents the number n . For example the string "11111" represents the number five. This is because we are working in unary, the way the input is structured will carry itself as the meaning of an integer. By this the Turing Machine will be recognizing whether this unary string is a member of the language of primes. So since in our machine there is no built-in arithmetic, it will work through the whole thing using tape symbols, movements, and as well as state transitions to determine the membership in this language.

In our tape alphabet it becomes critical for us using it for describing the language and how it is processed. Furthermore, to the symbol 1 which is represented in the unary number. The machine will be using symbols such as #, \$, and % to mark boundaries, divisor positions, and subtraction progress. These symbols are not in the input language but they are used by the machine to extend the tape so that it can be operated on. This next step for the Turing Machine will then translate the unary input into a format that will allow the modular division phase. Much of the computation involves transforming the original language string into the different configurations of tape symbols that still represent the same underlying integers.

In our initialization phase, the machine will prepare the unary input so that it can test whether it belongs to the prime-number language. Next, it will then scan to the end of the unary string and will place the separator symbol (#) to divide the input from the workspace it will be using during the computation. The original unary string will stay intact so that it will continue to stay in the number n . Then the machine will construct a divisor, which will also take the form of a unary string. In our machine it will start with the largest meaningful divisor, which is $n-1$, and the machine will then copy most of the input to create then the unary divisor string of that size. This will maintain the consistency with the language both of the number and the divisor which exists as a unary word on the tape. Then the symbols such as \$, and % will then be used to mark the regions of the tape but the main computation will still be centered on the manipulation of the unary string.

Next in the modulo loop phase, the machine will then test whether the unary input will belong to the language by simulating the division operation through repeated unary subtraction. So, since both the input and the divisor are unary strings, the subtraction is then carried out by the symbols, by doing symbol by symbol comparison and overwriting. Then the Turing Machine will then align the divisor word with the input words and will then attempt to remove the matching 1's. This will mimic the idea of checking in the case of whether one unary word will fit evenly into another. In the case that a full subtraction ends exactly at the separator (#), it will mean that the

divisor word has divided the input word with no remainder, so the input string will not belong to a prime-number language. Then in the case that the subtraction fails it is because the divisor is longer than what remains of the input, so then the divisor does not divide the unary string, and that the machine must then try the next smaller divisor.

Next, in the check-and-subtract phase, the machine will then decide whether the unary word should be accepted into the language. In the case that if a divisor divides the input exactly, then the input is rejected because the composite numbers are not part of the prime-number language. So, if the machine then reduces the divisor all the way down to 1 without finding a division that works, then the input unary string will then belong to the language of prime numbers, and the machine will accept. In the case that neither condition is reached, then the machine will decrease the divisor (which is just shortening the unary divisor string by one symbol), then resets symbols, and repeats the modulo process again.

In summary, the machine is not just performing an arithmetic operation but it is recognizing whether the original unary string is a member of a specific language which is the language of all unary prime numbers. Now by interpreting the unary strings as numbers and then using the additional tape symbols to manage the process, the Turing Machine will determine which unary words should be accepted and which should be rejected.